

CommerceOS API Reference

Oct 6, 2026 · @MoallimUnited

Overview

The CommerceOS backend is a JSON REST API. Every screen of the dashboard gets its data from it. The frontend never talks to Supabase directly and needs no Supabase key.

Environment	Base URL
Local	<code>http://localhost:3001</code>
Production (Vercel)	<code>https://amd-hackathon-backend.vercel.app</code> (to be confirmed at deploy)

Store it as one env variable in the frontend, e.g.

```
NEXT_PUBLIC_API_URL=http://localhost:3001 .
```

- **Auth:** none for now. The backend runs in demo mode and scopes every request to the demo store. When login is added, you will send `Authorization: Bearer <Supabase access token>`; no endpoint paths change.
- **CORS:** `http://localhost:3000` is allowed in development. In production only the deployed frontend URL is allowed, so call the API from the browser on that origin.
- **Content type:** send `Content-Type: application/json` on POST and PATCH. All responses are JSON.
- **Field naming:** all request and response fields are camelCase. All ids are UUID strings. All timestamps are ISO-8601 UTC strings.

Quick check that the API is up:

```
curl http://localhost:3001/health
```

```
{ "success": true, "status": "ok", "database": { "status": "ok", "latencyMs": 85 } }
```

Conventions

Every response has a `success` boolean. Check it (or the HTTP status) before reading data.

Single object

```
{ "success": true, "data": { "id": "..." } }
```

List (every list endpoint is paginated)

```
{
  "success": true,
  "data": [ { "id": "..." } ],
  "pagination": { "page": 1, "limit": 20, "total": 100, "totalPages": 5 }
}
```

Error

```
{ "success": false, "error": { "code": "PRODUCT_NOT_FOUND", "message":
  "Product not found" } }
```

Validation errors (400) also carry `details`, one entry per bad field, which you can show next to form inputs:

```
{
  "success": false,
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Request validation failed",
    "details": [ { "path": "email", "message": "Invalid email address" } ]
  }
}
```

Query parameters used across endpoints

Parameter	Format	Notes
<code>page</code>	integer ≥ 1	Default 1
<code>limit</code>	integer 1-100	Default 20 (categories default 100)
<code>sortBy</code>	field name	Allowed values are listed per endpoint
<code>sortOrder</code>	<code>asc</code> <code>desc</code>	Default <code>desc</code> unless noted
<code>search</code>	text	Case-insensitive substring match
Multi-value filters	comma-separated	e.g. <code>?status=pending,processing</code>
<code>from</code> , <code>to</code>	<code>YYYY-MM-DD</code> or ISO-8601	A date-only <code>to</code> includes that whole day

Date ranges (dashboard and analytics)

Send either a preset or explicit dates:

- `range=7d | 30d | 90d | 1y` (default `30d`)
- or `from=2026-09-01&to=2026-09-30`
- optional `interval=day | week | month` for charts. Default: `day` up to 31 days, `week` up to 120 days, `month` beyond.

Every dashboard and analytics response echoes the resolved period:

```
"period": { "range": "30d", "from": "2026-09-06T14:00:00.000Z", "to": "2026-10-06T14:00:00.000Z", "interval": "day" }
```

Values to know

- **Money** is a plain number in the store currency (USD for the demo store), e.g. `129.99`. Get the currency from `GET /api/store`.
- **Percentages** are numbers already multiplied by 100, e.g. `3.1` means 3.1%.
- **Stock status** comes in two forms on products and inventory: `stockStatus` is the display label (`"In Stock"`, `"Low Stock"`, `"Out of Stock"`) and `stockLevel` is the code (`in`, `low`, `out`) used in the `stock` filter.
- **KPI comparisons** use one shape everywhere: `{ "value": 13897.18, "previousValue": 12011.4, "changePercent": 15.7 }`. `previousValue` is the previous period of the same length. `changePercent` is `null` when the previous value was 0, so show "new" or "—" instead of a percentage.
- Nullable values are `null`, never missing.

Endpoint map

58 endpoints, grouped by the screen that uses them. All paths except `/health` start with `/api`.

Screen	Method	Path	Purpose
System	GET	<code>/health</code>	API and database status
Dashboard	GET	<code>/api/dashboard/summary</code>	KPI cards: revenue, orders, customers, conversion
Dashboard	GET	<code>/api/dashboard/revenue</code>	Revenue chart
Dashboard	GET	<code>/api/dashboard/orders</code>	Orders chart

Screen	Method	Path	Purpose
Dashboard	GET	/api/dashboard/top-products	Top-selling products
Dashboard	GET	/api/dashboard/recent-orders	Recent orders table
Customers	GET	/api/customers	Customer table
Customers	GET	/api/customers/:id	Customer detail + recent orders
Customers	POST	/api/customers	Create customer
Customers	PATCH	/api/customers/:id	Edit customer
Customers	DELETE	/api/customers/:id	Delete customer
Products	GET	/api/products	Product table
Products	GET	/api/products/:id	Product detail
Products	POST	/api/products	Create product (with opening stock)
Products	PATCH	/api/products/:id	Edit product
Products	DELETE	/api/products/:id	Delete product
Categories	GET	/api/categories	Category list / dropdown
Categories	GET	/api/categories/:id	Category detail
Categories	POST	/api/categories	Create category
Categories	PATCH	/api/categories/:id	Edit category
Categories	DELETE	/api/categories/:id	Delete category
Orders	GET	/api/orders	Order table
Orders	GET	/api/orders/:id	Order detail with line items
Orders	POST	/api/orders	Create order
Orders	PATCH	/api/orders/:id	Change status or customer
Inventory	GET	/api/inventory	Stock table
Inventory	GET	/api/inventory/alerts	Low / out-of-stock alerts
Inventory	GET	/api/inventory/history	Stock movement log

Screen	Method	Path	Purpose
Inventory	GET	/api/inventory/:productId	One product's stock + recent movements
Inventory	PATCH	/api/inventory/:productId	Set low-stock threshold
Inventory	POST	/api/inventory/restock	Add stock
Inventory	POST	/api/inventory/adjust	Correct stock (count, damage, return)
Analytics	GET	/api/analytics/overview	All headline metrics with comparison
Analytics	GET	/api/analytics/revenue	Revenue over time
Analytics	GET	/api/analytics/orders	Orders over time + status breakdown
Analytics	GET	/api/analytics/conversion	Sessions and conversion over time
Analytics	GET	/api/analytics/customers	Customer growth
Analytics	GET	/api/analytics/retention	Retention rates + cohort table
Analytics	GET	/api/analytics/categories	Revenue by category
Analytics	GET	/api/analytics/products	Product performance
Reviews	GET	/api/reviews	Review list
Reviews	GET	/api/reviews/:id	Review detail
Reviews	POST	/api/reviews	Create review
Reviews	PATCH	/api/reviews/:id	Edit review
Reviews	DELETE	/api/reviews/:id	Delete review
AI	GET	/api/ai/insights	Stored AI insights
AI	GET	/api/ai/insights/:id	Insight detail + its actions
AI	PATCH	/api/ai/insights/:id	Mark viewed / dismissed / actioned
AI	GET	/api/ai/actions	Suggested actions
AI	GET	/api/ai/actions/:id	Action detail
AI	PATCH	/api/ai/actions/:id/approve	Approve a pending action

Screen	Method	Path	Purpose
AI	PATCH	/api/ai/actions/:id/reject	Reject a pending action
AI	GET	/api/ai/analysis-runs	Analysis run log
AI	GET	/api/ai/analysis-runs/:id	Analysis run detail
Settings	GET	/api/store	Store profile + settings
Settings	PATCH	/api/store	Edit store name, currency, timezone
Settings	GET	/api/store/settings	Store settings
Settings	PATCH	/api/store/settings	Edit settings

Dashboard

Five endpoints feed the home screen. All accept the date-range parameters (`range` , `from` , `to` , `interval`).

GET /api/dashboard/summary

The KPI cards. Each metric is a comparison object against the previous period of the same length.

```
GET /api/dashboard/summary?range=30d
```

```
{
  "success": true,
  "data": {
    "period": { "range": "30d", "from": "...", "to": "...", "interval": "day" },
    "totalRevenue": { "value": 13897.18, "previousValue": 12011.4,
"changePercent": 15.7 },
    "totalOrders": { "value": 98, "previousValue": 90,
"changePercent": 8.89 },
    "totalCustomers": { "value": 160, "previousValue": 143,
"changePercent": 11.89 },
    "conversionRate": { "value": 3.1, "previousValue": 2.95,
"changePercent": 5.08 },
    "averageOrderValue": { "value": 157.92, "previousValue": 149.2,
"changePercent": 5.84 },
    "newCustomers": { "value": 17, "previousValue": 15,
"changePercent": 13.33 }
  }
}
```

`conversionRate.value` can be `null` when there were no store visits in the period.

GET /api/dashboard/revenue

Revenue chart. One point per bucket; empty buckets are included with 0, so you can plot the array as-is.

```
GET /api/dashboard/revenue?range=1y&interval=month
```

```
{
  "success": true,
  "data": {
    "period": { "range": "1y", "interval": "month", "from": "...", "to": "..." },
    "totalRevenue": 93563.06,
    "series": [
      { "date": "2025-10-01", "revenue": 3120.5, "orders": 21,
"averageOrderValue": 148.6 }
    ]
  }
}
```

`date` is the first day of the bucket (`YYYY-MM-DD`).

GET /api/dashboard/orders

Orders chart, same bucketing as revenue.

```
{
  "period": { ... },
  "totalOrders": 98,
  "series": [ { "date": "2026-10-05", "orders": 4, "completed": 3,
"cancelled": 0 } ]
}
```

GET /api/dashboard/top-products

Param	Values	Default
limit	1-50	5
sortBy	units revenue	units

```
{
  "period": { ... },
  "products": [
    { "productId": "...", "name": "Leather Card Wallet", "category":
"Accessories", "unitsSold": 85, "revenue": 3825, "ordersCount": 71 }
  ]
}
```

GET /api/dashboard/recent-orders

?limit=5 (1-50, default 5). Returns `data` as an array of order objects, the same shape as the [Orders](#) list (no pagination).

Customers

`ordersCount` and `totalSpent` are calculated live from orders, so they are always current and read-only.

GET /api/customers

Param	Values	Default
search	matches name, email, phone	—
status	active inactive	all
sortBy	name, email, orders, totalSpent, lastOrderAt, createdAt	createdAt

Param	Values	Default
sortOrder	asc desc	desc
page, limit		1, 20

GET /api/customers?search=john&sortBy=totalSpent&sortOrder=desc

```
{
  "success": true,
  "data": [
    {
      "id": "4b0c...",
      "firstName": "John",
      "lastName": "Smith",
      "name": "John Smith",
      "email": "john.smith@example.com",
      "phone": "+1-555-201-3344",
      "status": "active",
      "ordersCount": 7,
      "completedOrdersCount": 6,
      "totalSpent": 812.4,
      "firstOrderAt": "2026-01-10T09:12:00.000Z",
      "lastOrderAt": "2026-09-28T17:40:00.000Z",
      "createdAt": "...",
      "updatedAt": "..."
    }
  ],
  "pagination": { "page": 1, "limit": 20, "total": 1, "totalPages": 1 }
}
```

- `ordersCount` = orders that are not cancelled.
- `totalSpent` = total of the customer's completed orders.
- `status` is lowercase; capitalise it for display.

GET /api/customers/:id

Same object plus `recentOrders`: the customer's last 10 orders, in the [Orders](#) list shape.

POST /api/customers

```
{
  "firstName": "Jane",
  "lastName": "Doe",
  "email": "jane@example.com",
  "phone": "+1-555-0100",
  "status": "active"
}
```

Only `firstName` and `email` are required; `status` defaults to `active`. Returns 201 with the customer. An email already used in the store returns `409 CUSTOMER_EMAIL_EXISTS`.

PATCH /api/customers/:id

Send only the fields you change, e.g. `{ "status": "inactive" }`. Send `null` or `""` to clear `lastName` or `phone`.

DELETE /api/customers/:id

A customer with orders cannot be deleted by default; you get `409 CUSTOMER_HAS_ORDERS`. Offer "mark inactive" instead, or call `DELETE /api/customers/:id?force=true` to delete anyway (their orders stay, with no customer attached). Success returns `{ "id": "...", "deleted": true }`.

Products and categories

`unitsSold`, `revenue` and `stockStatus` are calculated from orders and inventory; they are read-only. Stock quantity is changed only through the [Inventory](#) endpoints, never through product PATCH.

GET /api/products

Param	Values	Default
<code>search</code>	matches name, SKU, category name	—
<code>category</code>	category slug (e.g. <code>shoes</code>) or id	all
<code>stock</code>	<code>in</code> , <code>low</code> , <code>out</code> (comma-separated)	all
<code>active</code>	<code>true</code> <code>false</code>	all
<code>minPrice</code> , <code>maxPrice</code>	number	—

Param	Values	Default
sortBy	name, price, stock, unitsSold, revenue, category, createdAt	createdAt
sortOrder	asc desc	desc

GET /api/products?category=shoes&stock=low,out&sortBy=unitsSold

```
{
  "success": true,
  "data": [
    {
      "id": "...",
      "name": "Everyday Running Shoes",
      "slug": "everyday-running-shoes",
      "sku": "SHO-RUN-01",
      "description": "...",
      "price": 129.99,
      "costPrice": 61.2,
      "active": true,
      "category": { "id": "...", "name": "Shoes", "slug": "shoes" },
      "stockQuantity": 4,
      "lowStockThreshold": 10,
      "stockStatus": "Low Stock",
      "stockLevel": "low",
      "unitsSold": 74,
      "revenue": 9619.26,
      "reviewCount": 12,
      "averageRating": 4.33,
      "createdAt": "...",
      "updatedAt": "..."
    }
  ],
  "pagination": { "page": 1, "limit": 20, "total": 1, "totalPages": 1 }
}
```

- `category` is `null` for uncategorised products. `averageRating` is `null` with no reviews.
- Stock status rule: `Out of Stock` when quantity is 0; `Low Stock` when quantity $\leq$ `lowStockThreshold`; otherwise `In Stock`.
- `unitsSold` and `revenue` count completed orders only, all time.

GET /api/products/:id

One product, same shape.

POST /api/products

```
{
  "name": "Trail Socks",
  "price": 14.99,
  "costPrice": 5,
  "categoryId": "...",
  "sku": "SP0-SCK-01",
  "description": "Merino trail socks",
  "active": true,
  "initialQuantity": 100,
  "lowStockThreshold": 15
}
```

Only `name` and `price` are required. Prices allow at most 2 decimals. `initialQuantity` (default 0) sets opening stock; `lowStockThreshold` defaults to the store setting. `slug` is optional and generated from the name. Returns 201 with the product.

PATCH /api/products/:id

Any of `name`, `slug`, `description`, `sku`, `price`, `costPrice`, `categoryId` (`null` to uncategorise), `active`. Changing `price` does not change past orders.

DELETE /api/products/:id

Permanently deletes the product and its stock history; past orders keep their line items. For "archive", use `PATCH { "active": false }` instead — we recommend that as the default UI action.

Categories

Method	Path	Notes
GET	<code>/api/categories</code>	<code>search</code> ; <code>sortBy</code> = <code>name</code> <code>slug</code> <code>createdAt</code> (default <code>name</code> , <code>asc</code>); <code>limit</code> default 100
GET	<code>/api/categories/:id</code>	
POST	<code>/api/categories</code>	<code>{ "name": "Shoes", "slug": "shoes", "description": "Footwear" }</code> — only <code>name</code> required

Method	Path	Notes
PATCH	/api/categories/:id	any of <code>name</code> , <code>slug</code> , <code>description</code>
DELETE	/api/categories/:id	its products become uncategorised

Category object:

```
{ "id": "...", "name": "Shoes", "slug": "shoes", "description": "Footwear",
  "productCount": 6, "createdAt": "...", "updatedAt": "..." }
```

Slugs are lowercase letters, numbers and single hyphens (`home-kitchen`). A slug already used in the store returns `409 CATEGORY_SLUG_EXISTS` .

Orders

Orders have four statuses: `pending`, `processing`, `completed`, `cancelled` . Revenue counts completed orders only.

GET /api/orders

Param	Values	Default
<code>search</code>	order number, customer name or email	—
<code>status</code>	comma-separated statuses	all
<code>customerId</code>	UUID	—
<code>from</code> , <code>to</code>	date range on order date	—
<code>minTotal</code> , <code>maxTotal</code>	number	—
<code>sortBy</code>	<code>orderedAt</code> , <code>total</code> , <code>orderNumber</code> , <code>status</code> , <code>customer</code>	<code>orderedAt</code>
<code>sortOrder</code>	<code>asc</code> <code>desc</code>	<code>desc</code>

```
GET /api/orders?status=completed&from=2026-10-01&to=2026-10-06
```

```
{
  "success": true,
  "data": [
    {
      "id": "...",
      "orderNumber": "ORD-100642",
```

```

    "status": "completed",
    "customer": { "id": "...", "name": "Aisha Khan", "email":
"aisha.khan@example.com" },
    "amount": 141.47,
    "subtotal": 128,
    "discount": 0,
    "tax": 10.24,
    "shipping": 0,
    "total": 141.47,
    "itemsCount": 2,
    "unitsCount": 3,
    "orderedAt": "2026-10-03T14:22:10.000Z",
    "createdAt": "...",
    "updatedAt": "..."
  }
],
"pagination": { "page": 1, "limit": 20, "total": 9, "totalPages": 1 }
}

```

- `amount` equals `total` ; use either for the "Amount" column.
- `customer` is `null` for guest orders or deleted customers — show "Guest".
- Show `orderNumber` to users, not `id` .

GET /api/orders/:id

The order plus its line items:

```

"items": [
  { "id": "...", "productId": "...", "productName": "Everyday Running Shoes",
"quantity": 1, "unitPrice": 129.99, "subtotal": 129.99 }
]

```

`productName` and `unitPrice` are the values at the time of the order. `productId` can be `null` if the product was later deleted.

POST /api/orders

```
{
  "customerId": "...",
  "items": [
    { "productId": "...", "quantity": 2 },
    { "productId": "...", "quantity": 1, "unitPrice": 19.5 }
  ],
  "status": "pending",
  "discount": 5,
  "tax": 4.2,
  "shipping": 7.99
}
```

Field	Required	Notes
<code>items</code>	yes	1-200 lines; <code>quantity</code> ≥ 1
<code>items[].unitPrice</code>	no	Defaults to the product's current price
<code>customerId</code>	no	Omit or <code>null</code> for a guest order
<code>status</code>	no	<code>pending</code> (default), <code>processing</code> or <code>completed</code>
<code>discount</code> , <code>tax</code> , <code>shipping</code>	no	Default 0
<code>orderNumber</code>	no	Generated as the next <code>ORD-<n></code>
<code>orderedAt</code>	no	ISO timestamp, defaults to now

The server calculates `subtotal` and `total` (`subtotal - discount + tax + shipping`) and deducts stock. Everything happens in one transaction: if any line fails, nothing is saved. Returns 201 with the full order including `items`.

Common errors: 409 `INSUFFICIENT_STOCK` (the message names the product and available quantity), 422 `PRODUCT_INACTIVE`, 404 `PRODUCT_NOT_FOUND`, 404 `CUSTOMER_NOT_FOUND`, 422 `INVALID_DISCOUNT`.

PATCH /api/orders/:id

```
{ "status": "completed" }
```

You can also send `{ "customerId": "..." }` or `{ "customerId": null }`. Returns the full order.

Allowed status changes — only offer these in the UI:

Current status	Can change to
pending	processing, completed, cancelled
processing	completed, cancelled
completed	cancelled (treated as a return)
cancelled	nothing — final

Cancelling puts the items back into stock automatically. Any other change returns `409 INVALID_STATUS_TRANSITION`.

Inventory

Every stock change — sale, cancellation, restock, adjustment — is recorded as a movement, so history is always complete. Stock can never go below 0.

GET /api/inventory

Param	Values	Default
search	product name, SKU, category	—
category	slug or id	all
stock	in, low, out (comma-separated)	all
sortBy	name, quantity, threshold, status, updatedAt	name
sortOrder	asc desc	asc

Inventory item:

```
{
  "productId": "...",
  "productName": "Smart Fitness Watch",
  "sku": "ELE-WCH-01",
  "active": true,
  "price": 149,
  "category": { "id": "...", "name": "Electronics", "slug": "electronics" },
  "quantity": 3,
  "lowStockThreshold": 8,
  "stockStatus": "Low Stock",
  "stockLevel": "low",
  "updatedAt": "..."
}
```

GET /api/inventory/alerts

Active products that are low or out of stock, lowest quantity first. Same item shape; paginated. Use `pagination.total` for an alert badge count.

GET /api/inventory/history

Param	Values
<code>productId</code>	UUID
<code>type</code>	<code>sale</code> , <code>restock</code> , <code>adjustment</code> , <code>return</code> , <code>cancellation</code> (comma-separated)
<code>from</code> , <code>to</code>	dates
<code>sortOrder</code>	<code>desc</code> (default) <code>asc</code>

```
{
  "id": "...",
  "productId": "...",
  "productName": "Smart Fitness Watch",
  "sku": "ELE-WCH-01",
  "type": "sale",
  "quantityChange": -2,
  "previousQuantity": 5,
  "newQuantity": 3,
  "referenceId": "<order id>",
  "note": "Order ORD-100640",
  "createdAt": "..."
}
```

`quantityChange` is signed: negative for sales and write-offs, positive for restocks, returns and cancellations. For `sale` and `cancellation`, `referenceId` is the order id, so you can link to the order.

GET /api/inventory/:productId

The inventory item plus `recentMovements` (the last 20).

PATCH /api/inventory/:productId

Set the low-stock threshold: `{ "lowStockThreshold": 12 }`. Returns the updated item.

POST /api/inventory/restock

```
{ "productId": "...", "quantity": 50, "note": "PO-1182" }
```

POST /api/inventory/adjust

Send exactly one of `quantityChange` (signed delta) or `newQuantity` (the counted total):

```
{ "productId": "...", "quantityChange": -3, "type": "adjustment", "note":  
  "Damaged in warehouse" }
```

```
{ "productId": "...", "newQuantity": 42, "note": "Monthly stock count" }
```

`type` is `adjustment` (default) or `return`. Both POST endpoints return 201:

```
{  
  "success": true,  
  "data": {  
    "movement": { "id": "...", "type": "restock", "quantityChange": 50,  
    "previousQuantity": 3, "newQuantity": 53 },  
    "inventory": { "productId": "...", "quantity": 53, "stockStatus": "In  
    Stock", "...": "..." }  
  }  
}
```

A change that would go below 0 returns `422 NEGATIVE_INVENTORY`; a change that leaves the quantity the same returns `422 INVALID_INVENTORY_CHANGE`.

Analytics

All analytics endpoints accept the date-range parameters and return a `period` object. Time series always include every bucket, with zeros where there was no activity.

Endpoint	data contains
GET /api/analytics/overview	One comparison object (value , previousValue , changePercent) for each of: revenue , orders , completedOrders , averageOrderValue , unitsSold , totalCustomers , newCustomers , purchasingCustomers , revenuePerCustomer , conversionRate , sessions , visitors , retentionRate , returningCustomerRate , repeatPurchaseRate
GET /api/analytics/revenue	totals { revenue , completedOrders , averageOrderValue , unitsSold }; series [{ date , revenue , completedOrders , averageOrderValue , unitsSold }]
GET /api/analytics/orders	totals { orders , averageOrderValue }; statusBreakdown { pending , processing , completed , cancelled }; series [{ date , orders , completed , cancelled }]
GET /api/analytics/conversion	totals { sessions , visitors , completedOrders , conversionRate }; series [{ date , sessions , completedOrders , conversionRate }]
GET /api/analytics/customers	totals { totalCustomers , newCustomers , growthRate , purchasingCustomers , revenuePerCustomer }; series [{ date , newCustomers , totalCustomers , activeCustomers , revenuePerCustomer }]
GET /api/analytics/retention	retentionRate , retainedCustomers , previousPeriodCustomers , returningCustomerRate , returningCustomers , repeatPurchaseRate , repeatCustomers , purchasingCustomers , cohorts (below). Defaults to range=1y
GET /api/analytics/categories	categories [{ categoryId , name , revenue , unitsSold , ordersCount , sharePercent }], highest revenue first

Endpoint	data contains
GET /api/analytics/products	products [{ productId, name, category, price, unitsSold, revenue, ordersCount, cost, grossProfit, marginPercent, reviewCount, averageRating, stockQuantity, stockStatus, stockLevel }]. Extra params: sortBy = revenue (default), units, profit, rating, name; limit 1-200, default 50

Retention cohorts, ready for a cohort heat-map (one row per cohort, one cell per month offset; offset 0 is always 100%):

```
"cohorts": [
  {
    "cohortMonth": "2026-03-01",
    "cohortSize": 12,
    "months": [
      { "monthOffset": 0, "customers": 12, "rate": 100 },
      { "monthOffset": 1, "customers": 4, "rate": 33.33 }
    ]
  }
]
```

Months with no returning customers are left out of `months` ; treat a missing offset as 0.

How the metrics are defined

Use these for tooltips and help text.

Metric	Definition
Revenue	Sum of order totals of completed orders
Orders	Orders that are not cancelled
Average order value	Revenue ÷ completed orders
Conversion rate	Completed orders ÷ unique store sessions × 100
Revenue per customer	Revenue ÷ customers with a completed order in the period
Customer growth rate	New customers ÷ customers at the start of the period × 100

Metric	Definition
Retention rate	% of customers who bought in the previous period and bought again in this one
Returning customer rate	% of this period's buyers who had bought before
Repeat purchase rate	% of this period's buyers with 2+ orders in the period
Category / product revenue	Line-item subtotals of completed orders (before order-level discount, tax and shipping), so category totals can be slightly below store revenue

Reviews, AI and store settings

Reviews

Method	Path	Notes
GET	<code>/api/reviews</code>	Filters: <code>productId</code> , <code>customerId</code> , <code>rating</code> , <code>minRating</code> , <code>maxRating</code> , <code>sentiment</code> (<code>positive</code> , <code>neutral</code> , <code>negative</code> , <code>unanalyzed</code> ; comma-separated), <code>search</code> (title and text). <code>sortBy</code> = <code>createdAt</code> <code>rating</code>
GET	<code>/api/reviews/:id</code>	
POST	<code>/api/reviews</code>	<code>{ "productId", "customerId"?, "rating": 1-5, "title"?, "reviewText"?, "sentiment"? }</code>
PATCH	<code>/api/reviews/:id</code>	any of <code>rating</code> , <code>title</code> , <code>reviewText</code> , <code>sentiment</code> , <code>customerId</code>
DELETE	<code>/api/reviews/:id</code>	

```
{
  "id": "...",
  "productId": "...",
  "product": { "id": "...", "name": "Everyday Running Shoes" },
  "customerId": "...",
  "customer": { "id": "...", "name": "John Smith", "email":
"john.smith@example.com" },
  "rating": 5,
  "title": "Perfect",
  "reviewText": "Exactly as described, would buy again.",
  "sentiment": "positive",
  "createdAt": "..."
}
```

`sentiment` is `null` for reviews not yet analysed (`?sentiment=unanalyzed`).

AI insights and actions

These endpoints only read stored records; AI generation comes in a later phase. Lists can be empty — design an empty state.

Method	Path	Notes
GET	<code>/api/ai/insights</code>	Filters: <code>type</code> , <code>priority</code> , <code>status</code> (comma-separated); <code>includeExpired=true</code> to show expired ones. Newest first
GET	<code>/api/ai/insights/:id</code>	Insight plus its linked <code>actions</code>
PATCH	<code>/api/ai/insights/:id</code>	<code>{ "status": "viewed" }</code> — <code>new</code> , <code>viewed</code> , <code>dismissed</code> , <code>actioned</code>
GET	<code>/api/ai/actions</code>	Filters: <code>status</code> , <code>insightId</code> , <code>type</code>
GET	<code>/api/ai/actions/:id</code>	
PATCH	<code>/api/ai/actions/:id/approve</code>	No body. Pending → approved
PATCH	<code>/api/ai/actions/:id/reject</code>	No body. Pending → rejected
GET	<code>/api/ai/analysis-runs</code>	Filters: <code>status</code> , <code>analysisType</code>
GET	<code>/api/ai/analysis-runs/:id</code>	

Insight:

```
{
  "id": "...",
  "type": "inventory",
  "priority": "critical",
  "title": "2 products out of stock, 5 running low",
  "summary": "...",
  "explanation": "...",
  "recommendation": "Restock the affected products to avoid lost sales.",
  "data": { },
  "status": "new",
  "generatedBy": "seed",
  "modelName": null,
  "generatedAt": "...",
  "expiresAt": "...",
  "createdAt": "..."
}
```

Values: `type` = `revenue`, `sales`, `inventory`, `customer`, `product`, `marketing`, `general`. `priority` = `low`, `medium`, `high`, `critical`.

Action:

```
{
  "id": "...",
  "insightId": "...",
  "type": "restock_product",
  "title": "Restock Smart Fitness Watch",
  "description": "...",
  "status": "pending",
  "parameters": { "productId": "...", "quantity": 50 },
  "result": null,
  "errorMessage": null,
  "approvedAt": null,
  "executedAt": null,
  "createdAt": "...",
  "updatedAt": "..."
}
```

Show Approve and Reject only when `status` is `pending`. Approving records the decision; it does not run anything yet. Approving or rejecting a non-pending action returns `409 INVALID_ACTION_STATE`.

Store and settings

`GET /api/store`:

```
{
  "id": "...",
  "name": "CommerceOS Demo Store",
  "slug": "commerceos-demo",
  "currency": "USD",
  "timezone": "UTC",
  "createdAt": "...",
  "updatedAt": "...",
  "settings": { "aiInsightsEnabled": true, "lowStockDefaultThreshold": 10,
  "persisted": true, "updatedAt": "..." }
}
```

Method	Path	Body
PATCH	/api/store	any of <code>name</code> , <code>slug</code> , <code>currency</code> (3-letter code, e.g. <code>EUR</code>), <code>timezone</code> (IANA, e.g. <code>Asia/Dubai</code>)
GET	/api/store/settings	— returns the <code>settings</code> object
PATCH	/api/store/settings	any of <code>aiInsightsEnabled</code> (boolean), <code>lowStockDefaultThreshold</code> (integer ≥ 0)

Call `GET /api/store` once at app start and use `currency` and `timezone` to format money and dates.

Errors and integration tips

Branch on `error.code`, not on the message; messages are human-readable and may change.

HTTP	Code	When	Suggested UI
400	<code>VALIDATION_ERROR</code>	Bad field, query or body	Show <code>details[].message</code> next to each <code>details[].path</code> field
400	<code>INVALID_JSON</code>	Malformed request body	Bug — log it
400	<code>INVALID_DATE_RANGE</code> , <code>INVALID_DATE</code> , <code>INVALID_INTERVAL</code>	Bad <code>from / to / interval</code>	Reset the date picker

HTTP	Code	When	Suggested UI
403	CORS_ORIGIN_DENIED	Frontend origin not allowed	Config issue — tell the backend team
404	CUSTOMER_NOT_FOUND , PRODUCT_NOT_FOUND , CATEGORY_NOT_FOUND , ORDER_NOT_FOUND , REVIEW_NOT_FOUND , AI_INSIGHT_NOT_FOUND , AI_ACTION_NOT_FOUND , STORE_NOT_FOUND	Id doesn't exist	"Not found" page
404	ROUTE_NOT_FOUND	Wrong URL	Bug — check the path
409	INSUFFICIENT_STOCK	Order exceeds available stock	Show the message; it names the product and quantity left
409	INVALID_STATUS_TRANSITION	Disallowed order status change	Refresh the order
409	CUSTOMER_EMAIL_EXISTS , CATEGORY_SLUG_EXISTS , STORE_SLUG_EXISTS , ORDER_NUMBER_EXISTS	Duplicate value	Inline error on that field
409	CUSTOMER_HAS_ORDERS	Deleting a customer with orders	Offer "Mark inactive"
409	INVALID_ACTION_STATE	Action already decided	Refresh the action
422	NEGATIVE_INVENTORY , INVALID_INVENTORY_CHANGE , PRODUCT_INACTIVE , INVALID_DISCOUNT	Business rule broken	Show the message
500	DATABASE_NOT_MIGRATED	Backend setup incomplete	Tell the backend team
500 / 503	DATABASE_ERROR , DATABASE_UNAVAILABLE , INTERNAL_ERROR	Server problem	Generic error + retry

A fetch helper

```

const API_URL = process.env.NEXT_PUBLIC_API_URL ?? 'http://localhost:3001';

export class ApiError extends Error {
  constructor(public status: number, public code: string, message: string,
    public details?: { path: string; message: string }[]) {
    super(message);
  }
}

export interface Pagination { page: number; limit: number; total: number;
totalPages: number }
export interface Metric { value: number | null; previousValue: number | null;
changePercent: number | null }

type Query = Record<string, string | number | boolean | string[] |
undefined>;

function toQuery(params?: Query) {
  if (!params) return '';
  const q = new URLSearchParams();
  for (const [key, value] of Object.entries(params)) {
    if (value === undefined || value === '') continue;
    q.set(key, Array.isArray(value) ? value.join(',') : String(value));
  }
  const s = q.toString();
  return s ? `?${s}` : '';
}

async function request<T>(method: string, path: string, options: { query?:
Query; body?: unknown } = {}) {
  const res = await fetch(`${API_URL}${path}${toQuery(options.query)}`, {
    method,
    headers: options.body ? { 'Content-Type': 'application/json' } :
undefined,
    body: options.body ? JSON.stringify(options.body) : undefined,
  });
  const json = await res.json();
  if (!json.success) {
    const e = json.error ?? {};
    throw new ApiError(res.status, e.code ?? 'UNKNOWN', e.message ?? 'Request
failed', e.details);
  }
  return json as { data: T; pagination?: Pagination };
}

```

```
export const api = {
  get: <T>(path: string, query?: Query) => request<T>('GET', path, { query
  })),
  post: <T>(path: string, body: unknown) => request<T>('POST', path, { body
  })),
  patch: <T>(path: string, body: unknown) => request<T>('PATCH', path, { body
  })),
  delete: <T>(path: string, query?: Query) => request<T>('DELETE', path, {
  query })),
};
```

Usage:

```
const { data: kpis } = await api.get<Record<string, Metric>>
('/api/dashboard/summary', { range: '30d' });
const { data: products, pagination } = await api.get('/api/products', {
category: 'shoes', stock: ['low', 'out'], page: 1 });
await api.patch(`/api/orders/${orderId}`, { status: 'completed' });
```

Tips

- Drive table pages from `pagination.totalPages`; requesting a page past the end returns an empty `data` array, not an error.
- Debounce `search` inputs by about 300 ms; every keystroke is a database query.
- After a create or update, use the returned object to update local state; it already includes recalculated fields such as `totalSpent` and `stockStatus`.
- After creating or cancelling an order, refetch inventory and dashboard data: stock and revenue change server-side.
- Login will later add one header, `Authorization: Bearer <token>`, inside `request()`. Nothing else in your code needs to change.